

• bitácora técnica — actualizada 2026-09-29

Videovigilancia con visión por computadora, de cero a cámara real

Detección de placas, rostros y comportamiento anómalo sobre una Hikvision real, corriendo en GPU, con lista negra, retención de datos y una vista en vivo — para el equipo, esto es dónde vamos y qué falta antes de presentarlo.

● Placas — en cámara real

● Rostros — en cámara real

● Movimiento anómalo — funcionando

● 2 cámaras a la vez

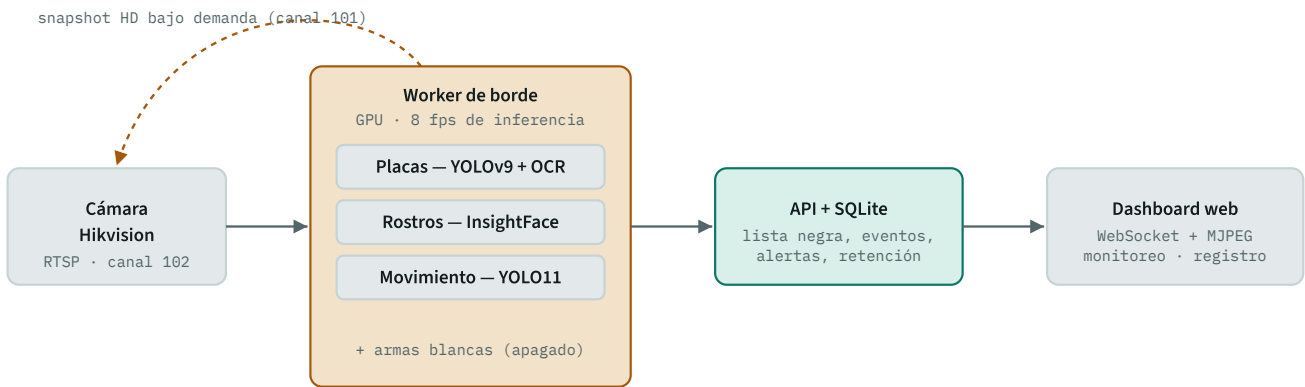
● Armas blancas — desactivado

01 Cómo funciona

Tres piezas independientes que se comunican por HTTP y WebSocket: la cámara nunca le habla directo al navegador.

La **Hikvision** manda su sub-stream (1280×720, canal 102) por RTSP al **worker de borde**: un proceso en Python que lee el video, corre tres detectores en la GPU y decide qué merece convertirse en un evento. Ningún detector habla directo con la base de datos: arma un `DetectionEvent` y se lo pasa a la **API**, que es quien decide la severidad cruzando contra la lista negra y quien guarda todo en SQLite.

El **dashboard** (una sola página, sin framework) se conecta por WebSocket para ver eventos y alertas en tiempo real, y recibe el video ya anotado del worker como un flujo MJPEG — el navegador nunca toca la cámara directamente, porque vive en una red a la que no llega.



02 Estado por módulo

Los cuatro frentes que pidieron revisar, con lo que ya funciona y lo que falta antes de darlo por cerrado.

Placas

funcionando

35/35

10

LECTURAS CON ÁNGULO

LECTURAS OCR/TRACK

YA FUNCIONA

- ✓ YOLOv9 detecta y un OCR entrenado solo con placas lee en menos de 1 ms, con consenso de varias lecturas por vehículo
- ✓ 35/35 lecturas correctas con la cámara de lado, desde arriba, lejos y con poca luz (antes 20/35)
- ✓ Une la placa cuando el OCR la parte en trozos y corrige por posición (A8C-I23 → ABC-123)
- ✓ Voto carácter por carácter entre lecturas: recupera la placa aunque ningún frame la lea completa
- ✓ Re-escaneo retroactivo y vigencia en la lista negra
- ✓ Evidencia en 3200×1800 (antes 1280×720)

FALTA

- Probar con la cámara apuntando a calle/cochera (hoy el encuadre es interior)
- Afinar el OCR con capturas de placas de la cámara real

Rostros

funcionando

0.50

512d

UMBRAL DE MATCH

EMBEDDING ARCFACE

YA FUNCIONA

- ✓ InsightFace (buffalo_l) detecta y calcula embedding en GPU
- ✓ Calidad por tamaño, pose (5 puntos faciales) y nitidez; plantilla con el promedio de las 3 mejores vistas
- ✓ Evidencia HD: recorta y vuelve a calcular el embedding sobre la foto de alta resolución
- ✓ Sin lista negra: nunca se guarda el embedding de nadie (dato biométrico, LFPDPPP)

FALTA

- **0.50 es un valor de fábrica:** nadie lo calibró contra una persona real todavía
- Dar de alta una foto real y medir falsos positivos/negativos

Movimiento anómalo

funcionando

2.5

3/5

ALTURAS/SEG UMBRAL

FRAMES CONFIRMACIÓN

Armas blancas

desactivado

nano→small

0/1

MODELO PROBADO

DETECTADO EN PRUEBA REAL

YA FUNCIONA

- ✓ Mide velocidad de cada persona seguida, normalizada por su propio tamaño en pantalla
- ✓ Descarta los saltos del tracker (cambio de identidad, oclusión) en vez de contarlos como velocidad
- ✓ Cero falsos positivos en 30s de actividad normal (dos personas caminando)

FALTA

- **Nunca se probó con un movimiento brusco real** — el 2.5 es una estimación
- Decidir si un forcejeo real dispara con ese umbral, o hace falta afinarlo

QUÉ SE INTENTÓ

- ✓ YOLO11 sobre clases de COCO (cuchillo, tijeras, bate)
- ✓ Se subió de modelo nano a small y la resolución de 640 a 960px

POR QUÉ ESTÁ APAGADO

- Probado con un cuchillo real en mano, en penumbra: **no lo detectó**
- COCO no tiene clase de arma de fuego; para eso hace falta entrenar un modelo propio

03 Cómo probarlo

De repo limpio a viendo la cámara en el navegador. En una laptop con GPU NVIDIA toma unos 10 minutos.

01 Clonar y crear el entorno

El venv es obligatorio: instalar todo con el Python del sistema es justo el error que nos hizo correr sin GPU sin darnos cuenta.

```
python -m venv venv
venv\Scripts\activate
pip install torch==2.13.0+cu126 torchvision==0.28.0+cu126 --index-url https://download.pytorch.org/whl/cu126
pip install -r requirements.txt
```

02 Configurar la cámara

Copia `.env.example` a `.env`. Con webcam sirve el valor por defecto; con la Hikvision, usa `tools/probe_camara.py` para encontrar la URL RTSP correcta.

03 Diagnóstico de fuente, sin cargar modelos

Si aquí los fps son inestables, ningún modelo lo va a arreglar — hay que resolverlo antes de seguir.

```
python -m edge.worker --diagnostico
```

04 Levantar la plataforma

Dos procesos, cada uno en su terminal (o con los `.bat` incluidos, que siempre usan el Python correcto).

```
iniciar_api.bat
iniciar_worker.bat
```

05 Probar cada módulo desde el navegador

Entra a `http://localhost:8000`. Para probar de verdad: pásate frente a la cámara (rostro), muestra una placa, haz un movimiento brusco y mira si sale el aviso amarillo, da de alta algo en "Lista negra" para ver si el re-escaneo encuentra apariciones pasadas, y búscalo en Registro.

06 Correr las pruebas automatizadas

No necesitan cámara ni GPU: tracking, anti-falsos-positivos, lectura de placas, rostros, envío con la API caída y la API completa contra una base de datos temporal.

```
python tests/correr_todas.py # 87 pruebas en 9 archivos
```

04 Detalles extra

Lo que no cabe en una demo pero seguro preguntan en la evaluación del reto.

Privacidad y retención (LFPDPPP)

El sistema borra evidencia sola. Las **fotos** de eventos normales duran 7 días; el **texto** del evento (para responder "¿pasó este coche?") dura 30 días; una **alerta** confirmada, un año. El dato más sensible —el embedding facial— **solo se guarda si la persona ya estaba en la lista negra**; si no coincidió, se descarta en memoria y nunca toca el disco. Es la misma regla que hace posible el re-escaneo retroactivo sin crear una base de datos biométrica de todo el que pasó por la cámara.

QUÉ	DÍAS	POR QUÉ
Fotos de eventos	7	Verificar un caso reciente
Eventos (texto)	30	Responder "¿pasó tal cosa?"
Alertas confirmadas	365	Historial para auditoría
Embeddings faciales	7*	*Solo si hubo coincidencia

GPU	RTX 4050, 6 GB
FPS de cámara	20.7 medidos
FPS de inferencia	8 (a propósito)
3 detectores/frame	~50 ms de 125 ms
Sub-stream	1280×720
Snapshot evidencia	3200×1800

La GPU sobra: el límite hoy no es cómputo, es cuántas cámaras y qué tan preciso quieras el modelo. Ver nota abajo sobre CPU vs GPU.

La trampa que ya nos costó tiempo: CPU vs GPU

Todo el stack (torch, ultralytics, onnxruntime...) se puede instalar dos veces sin darte cuenta: una en el venv del proyecto con soporte CUDA, y otra en el Python global de Windows sin GPU, si alguna vez corriste `pip install` con una terminal donde el venv no estaba activado. Los tres detectores en CPU tardan 10-20x más — la GPU nunca fue el problema, era qué intérprete estaba corriendo. Por eso los `.bat` del paso 4 apuntan siempre al Python del venv, y por eso el worker ahora grita en consola si detecta que va a correr sin GPU.